

STUDENT RESULT PORTAL

Project Presentation

Python • Django • MySQL • HTML/CSS/JavaScript

Project Purpose	A web-based portal for centralized management of student academic information and examination results.
Primary Users	Admin, Teacher and Student
Architecture	Django MVT architecture with Django ORM, authentication and MySQL database.

Problem Statement

Managing student records and examination results manually can lead to duplicated data, calculation errors, slow result access and difficulty controlling who can view or modify academic information. The Student Result Portal provides a centralized web application where authorized users can manage students, teachers, classes, subjects and results, while students can securely access their own academic information.

Project Objectives

- Centralize student and academic-result data in a MySQL database.
- Provide secure login and role-based access for Admin, Teacher and Student.
- Allow teachers to enter, view and update student results.
- Allow students to view their own academic results and related information.
- Provide a maintainable Django-based structure using reusable apps, models, views, templates and URLs.

Technology Stack & System Architecture

Layer	Technology / Tool	Purpose
Backend	Python 3.13, Django 6.x	Application logic, routing, authentication and MVT structure
Database	MySQL / MySQL Workbench	Persistent storage of academic and user data
ORM	Django ORM & Migrations	Models, queries and database schema management
Frontend	HTML, CSS, JavaScript, Django Templates	Dashboards, forms and result presentation
Testing/API	Django tests, Postman	Application/API testing and verification
Version Control	Git & GitHub	Source control, incremental commits and project sharing

Architecture / Data Flow

USER Admin / Teacher / Student	Django URL Router Requests are mapped to application views	Views + Services Business rules, validation and access control
↓	↓	↓
Django Templates HTML/CSS/JavaScript dashboards	Django ORM / Models Database operations	MySQL Academic data and results

ER Diagram Overview

The database is organized around users and academic entities. User accounts are associated with Student or Teacher profiles. Classes, Subjects and Results are linked through the relevant academic relationships. Result records store student performance information such as marks and calculated result details. The project documentation contains the actual ER diagram generated for the implemented MySQL database schema.

Core relationships: User → Student/Teacher; Class → Students; Subject → Results; Student → Results; Teacher → permitted academic operations.

Features Implemented

Module / Feature	Implementation
Admin Module	Manage users and academic master data through Django Admin; manage classes, subjects, students, teachers and result-related data.
Authentication	Login/logout functionality using Django authentication with protected pages.
Role-Based Access	Separate Admin, Teacher and Student access and dashboards; unauthorized role access is restricted.
Teacher Dashboard	Teacher can view the dashboard and enter, view and update student results according to permitted operations.
Student Dashboard	Student can securely view personal academic information and own results.
Result Management	Marks/result information is stored in MySQL and displayed through Django templates.
Database Integration	Django ORM and migrations connect the application to MySQL; database was verified using MySQL Workbench.
Documentation & Version Control	README, API/Postman documentation, ER diagram, GitHub repository and incremental commits prepared for submission.

Application Structure

- **accounts** – authentication, login and role-related access.
- **classes** – class/academic-class management.
- **students** – student profile and academic information.
- **teachers** – teacher-related functionality.
- **subjects** – subject information.
- **results** – result/marks management and display.
- **frontend/templates** – user-facing dashboards and pages.

Security / Access Control

The application uses authenticated sessions and role checks so that users are directed to the appropriate dashboard. Students are restricted to their own academic information, while teacher and administrative functions are protected from unauthorized access.

Challenges Faced, Solutions & Reflection

Challenge	Solution / Learning
Django environment / import errors	Verified the virtual environment and Django installation before running the project.
Database migrations not detecting models	Checked INSTALLED_APPS, model locations, migration state and database configuration.
Model relationship conflict	Resolved duplicate Student model / reverse-accessor conflicts by consolidating the model relationship and using appropriate related names.
Git repository indexing issue	Removed the unintended nested repository structure, initialized the correct project repository and continued with incremental commits.
Result update URL error	Corrected the template URL so the result ID is passed correctly to the update-result route.
Template / URL configuration issues	Validated urlpatterns and used Django checks/runserver testing to locate and correct routing problems.

Short Reflection

Developing the Student Result Portal provided practical experience in building a complete web application with Django and MySQL. The project strengthened my understanding of MVT architecture, database relationships, migrations, authentication, role-based authorization, template-based frontend development and Git/GitHub workflow. Debugging migration, routing, model and template issues also improved my ability to trace errors systematically rather than changing code without identifying the underlying cause. The completed portal demonstrates how a structured web application can organize academic data and provide different functionality to administrators, teachers and students.

Project Deliverables

- Source code maintained in GitHub with incremental commit history.
- README.md with project setup and usage information.
- MySQL schema.sql and seed data documentation.
- ER diagram and API/Postman documentation.
- Screenshots / demonstration evidence as required for submission.
- Project presentation exported as PDF.

END OF PROJECT PRESENTATION